

Министерство науки и высшего образования Российской Федерации

Пензенский государственный университет

Кафедра «Вычислительная техника»

ОТЧЁТ

по лабораторной работе №3

по курсу «Логика и основы алгоритмизации в инженерных задачах»

на тему «Динамические списки»

Выполнили:

студенты группы 24ВВВ3

Задоркин Тимофей

Комиссаров Андрей

Приняли:

Юрова О.В.

Деев М.В.

Пенза 2025

Цель работы – Освоить принципы работы с динамическими структурами данных на примере односвязных списков, включая их создание, манипулирование элементами и организацию памяти.

Лабораторное задание:

1. Реализовать приоритетную очередь, путём добавления элемента в список в соответствии с приоритетом объекта (т.е. объект с большим приоритетом становится перед объектом с меньшим приоритетом).
2. * На основе приведенного кода реализуйте структуру данных *Очередь*.
3. * На основе приведенного кода реализуйте структуру данных *Стек*.

Задание 1

```
#define _CRT_SECURE_NO_WARNINGS
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

struct node {
    char inf[256];
    int priority;
    struct node* next;
};

struct node* head = NULL;
struct node* last = NULL;
char find_el[256];

struct node* get_struct(void);
struct node* find_by_name(char* name);
struct node* find_by_priority(int priority);
void find_menu(void);
void spstore(void);
void review(void);
void del_by_name(char* name);
void del_by_priority(int priority);
void del_menu(void);
struct node* dequeue(void);
void free_list(void);

struct node* get_struct(void)
{
    struct node* p = NULL;
    char s[256];
    int priority;

    if ((p = (struct node*)malloc(sizeof(struct node))) == NULL)
    {
        printf("Ошибка при распределении памяти\n");
        exit(1);
    }

    printf("Введите название объекта: \n");
    scanf("%s", s);
    if (*s == 0)
    {
        printf("Запись не была произведена\n");
        free(p);
        return NULL;
    }
}
```

```

    printf("Введите приоритет объекта (целое число, чем меньше - тем выше
приоритет): \n");
    scanf("%d", &priority);

    strcpy(p->inf, s);
    p->priority = priority;
    p->next = NULL;

    return p;
}

void spstore(void)
{
    struct node* p = NULL;
    struct node* current = NULL;
    struct node* prev = NULL;

    p = get_struct();
    if (p == NULL) {
        return;
    }

    if (head == NULL)
    {
        head = p;
        last = p;
        return;
    }

    current = head;
    prev = NULL;

    while (current != NULL && current->priority <= p->priority)
    {
        prev = current;
        current = current->next;
    }

    if (prev == NULL)
    {
        p->next = head;
        head = p;
    }
    else
    {
        prev->next = p;
        p->next = current;

        if (current == NULL)
        {
            last = p;
        }
    }
}

void review(void)
{
    struct node* struc = head;
    if (head == NULL)
    {
        printf("Список пуст\n");
        return;
    }

    printf("Содержимое списка (элементы с высшим приоритетом first):\n");
    while (struc)

```

```

    {
        printf("Имя - %s, Приоритет - %d\n", struc->inf, struc->priority);
        struc = struc->next;
    }
}

struct node* find_by_name(char* name)
{
    struct node* struc = head;
    if (head == NULL)
    {
        printf("Список пуст\n");
        return NULL;
    }

    while (struc)
    {
        if (strcmp(name, struc->inf) == 0)
        {
            return struc;
        }
        struc = struc->next;
    }

    printf("Элемент не найден\n");
    return NULL;
}

struct node* find_by_priority(int priority)
{
    struct node* struc = head;
    struct node* found = NULL;
    int count = 0;

    if (head == NULL)
    {
        printf("Список пуст\n");
        return NULL;
    }

    printf("Найденные элементы с приоритетом %d:\n", priority);
    while (struc)
    {
        if (struc->priority == priority)
        {
            printf("Имя - %s, Приоритет - %d\n", struc->inf, struc->priority);
            found = struc;
            count++;
        }
        struc = struc->next;
    }

    if (count == 0)
    {
        printf("Элементы с приоритетом %d не найдены\n", priority);
        return NULL;
    }

    return found;
}

void find_menu(void)
{
    int choice;
    char name[256];
    int priority;
    struct node* temp;

```

```

printf("\nПоиск элементов:\n");
printf("1. Поиск по имени\n");
printf("2. Поиск по приоритету\n");
printf("Выберите тип поиска: ");
scanf("%d", &choice);

switch (choice) {
case 1:
    printf("Введите имя для поиска: ");
    scanf("%s", name);
    temp = find_by_name(name);
    if (temp != NULL) {
        printf("Найден: %s с приоритетом %d\n", temp->inf, temp->priority);
    }
    break;
case 2:
    printf("Введите приоритет для поиска: ");
    scanf("%d", &priority);
    find_by_priority(priority);
    break;
default:
    printf("Неверный выбор\n");
}
}

void del_by_name(char* name)
{
    struct node* struc = head;
    struct node* prev = NULL;
    int flag = 0;

    if (head == NULL)
    {
        printf("Список пуст\n");
        return;
    }

    while (struc != NULL)
    {
        if (strcmp(name, struc->inf) == 0)
        {
            flag = 1;

            if (prev == NULL)
            {
                head = struc->next;
                if (head == NULL)
                {
                    last = NULL;
                }
                printf("Удален: %s с приоритетом %d\n", struc->inf, struc-
>priority);
                free(struc);
                struc = head;
            }
            else
            {
                prev->next = struc->next;
                if (struc->next == NULL)
                {
                    last = prev;
                }
                printf("Удален: %s с приоритетом %d\n", struc->inf, struc-
>priority);
                free(struc);
                struc = prev->next;
            }
        }
        prev = struc;
        struc = struc->next;
    }
}

```

```

        }
    }
    else
    {
        prev = struc;
        struc = struc->next;
    }
}

if (flag == 0)
{
    printf("Элемент не найден\n");
}
}

void del_by_priority(int priority)
{
    struct node* struc = head;
    struct node* prev = NULL;
    struct node* to_delete = NULL;
    int flag = 0;

    if (head == NULL)
    {
        printf("Список пуст\n");
        return;
    }

    while (struc != NULL)
    {
        if (struc->priority == priority)
        {
            flag = 1;

            if (prev == NULL)
            {
                head = struc->next;
                if (head == NULL)
                {
                    last = NULL;
                }
                printf("Удален: %s с приоритетом %d\n", struc->inf, struc-
>priority);
                free(struc);
                struc = head;
            }
            else
            {
                prev->next = struc->next;
                if (struc->next == NULL)
                {
                    last = prev;
                }
                printf("Удален: %s с приоритетом %d\n", struc->inf, struc-
>priority);
                free(struc);
                struc = prev->next;
            }
        }
        else
        {
            prev = struc;
            struc = struc->next;
        }
    }

    if (flag == 0)

```

```

    {
        printf("Элементы с приоритетом %d не найдены\n", priority);
    }
    else
    {
        printf("Удалены все элементы с приоритетом %d\n", priority);
    }
}

void del_menu(void)
{
    int choice;
    char name[256];
    int priority;

    printf("\nУдаление элементов:\n");
    printf("1. Удаление по имени\n");
    printf("2. Удаление по приоритету\n");
    printf("Выберите тип удаления: ");
    scanf("%d", &choice);

    switch (choice) {
    case 1:
        printf("Введите имя для удаления: ");
        scanf("%s", name);
        del_by_name(name);
        break;
    case 2:
        printf("Введите приоритет для удаления: ");
        scanf("%d", &priority);
        del_by_priority(priority);
        break;
    default:
        printf("Неверный выбор\n");
    }
}

struct node* dequeue(void)
{
    if (head == NULL) {
        printf("Очередь пуста\n");
        return NULL;
    }

    struct node* temp = head;
    head = head->next;

    if (head == NULL) {
        last = NULL;
    }

    temp->next = NULL;
    return temp;
}

void free_list(void)
{
    struct node* current = head;
    struct node* next;

    while (current != NULL) {
        next = current->next;
        free(current);
        current = next;
    }

    head = NULL;

```

```

        last = NULL;
    }

    int main()
    {
        system("chcp 1251");
        int choice;
        struct node* temp;

        printf("Приоритетная очередь (чем МЕНЬШЕ число приоритета - тем ВЫШЕ приоритет)\n");

        while (1) {
            printf("\nМеню:\n");
            printf("1. Добавить элемент\n");
            printf("2. Просмотреть все элементы\n");
            printf("3. Найти элемент\n");
            printf("4. Удалить элемент\n");
            printf("5. Извлечь элемент с высшим приоритетом\n");
            printf("6. Выход\n");
            printf("Выберите действие: ");
            scanf("%d", &choice);

            switch (choice) {
                case 1:
                    spstore();
                    break;
                case 2:
                    review();
                    break;
                case 3:
                    find_menu();
                    break;
                case 4:
                    del_menu();
                    break;
                case 5:
                    temp = dequeue();
                    if (temp != NULL) {
                        printf("Извлечен: %s с приоритетом %d\n", temp->inf, temp->priority);
                        free(temp);
                    }
                    break;
                case 6:
                    free_list();
                    printf("Выход...\n");
                    return 0;
                default:
                    printf("Неверный выбор\n");
            }
        }

        return 0;
    }
}

```

Результат работы программы

Результат работы программы показан на рисунках 1.1 – 1.5


```
Текущая кодовая страница: 1251
Приоритетная очередь (чем МЕНЬШЕ число приоритета - тем ВЫШЕ приоритет)

Меню:
1. Добавить элемент
2. Просмотреть все элементы
3. Найти элемент
4. Удалить элемент
5. Извлечь элемент с высшим приоритетом
6. Выход
Выберите действие: 1
Введите название объекта:
Игры
Введите приоритет объекта (целое число, чем меньше - тем выше приоритет):
3

Меню:
1. Добавить элемент
2. Просмотреть все элементы
3. Найти элемент
4. Удалить элемент
5. Извлечь элемент с высшим приоритетом
6. Выход
Выберите действие: 1
Введите название объекта:
Видеоигры
Введите приоритет объекта (целое число, чем меньше - тем выше приоритет):
1
```

Рисунок 1.1

```
Меню:
1. Добавить элемент
2. Просмотреть все элементы
3. Найти элемент
4. Удалить элемент
5. Извлечь элемент с высшим приоритетом
6. Выход
Выберите действие: 2
Содержимое списка:
Имя - Видеоигры, Приоритет - 1
Имя - Игры, Приоритет - 3
```

Рисунок 1.2

```
Меню:
1. Добавить элемент
2. Просмотреть все элементы
3. Найти элемент
4. Удалить элемент
5. Извлечь элемент с высшим приоритетом
6. Выход
Выберите действие: 3

Поиск элементов:
1. Поиск по имени
2. Поиск по приоритету
Выберите тип поиска: 1
Введите имя для поиска: Игры
Найден: Игры с приоритетом 3
```

Рисунок 1.3

```
Меню:
1. Добавить элемент
2. Просмотреть все элементы
3. Найти элемент
4. Удалить элемент
5. Извлечь элемент с высшим приоритетом
6. Выход
Выберите действие: 4

Удаление элементов:
1. Удаление по имени
2. Удаление по приоритету
Выберите тип удаления: 2
Введите приоритет для удаления: 1
Удален: Видеоигры с приоритетом 1
Удалены все элементы с приоритетом 1
```

Рисунок 1.4

```

Меню:
1. Добавить элемент
2. Просмотреть все элементы
3. Найти элемент
4. Удалить элемент
5. Извлечь элемент с высшим приоритетом
6. Выход
Выберите действие: 2
Содержимое списка:
Имя - Видеоигры, Приоритет - 1
Имя - Игры, Приоритет - 3

Меню:
1. Добавить элемент
2. Просмотреть все элементы
3. Найти элемент
4. Удалить элемент
5. Извлечь элемент с высшим приоритетом
6. Выход
Выберите действие: 5
Извлечен: Видеоигры с приоритетом 1

Меню:

```

Рисунок 1.5

Задание 2

```

#define _CRT_SECURE_NO_WARNINGS
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

struct node {
    char inf[256];
    struct node* next;
};

struct node* head = NULL;
struct node* last = NULL;
char find_el[256];

struct node* get_struct(void);
struct node* find(char* name);
void enqueue(void);
void dequeue(void);
void review(void);
void del(char* name);

struct node* get_struct(void)
{
    struct node* p = NULL;
    char s[256];

    if ((p = (struct node*)malloc(sizeof(struct node))) == NULL)
    {
        printf("Ошибка при распределении памяти\n");
        exit(1);
    }

    printf("Введите название объекта: \n");
    scanf("%s", s);
    if (*s == 0)
    {

```

```

        printf("Запись не была произведена\n");
        free(p);
        return NULL;
    }
    strcpy(p->inf, s);
    p->next = NULL;

    return p;
}

void enqueue(void)
{
    struct node* p = NULL;
    p = get_struct();

    if (p == NULL) {
        return;
    }

    if (head == NULL)
    {
        head = p;
        last = p;
    }
    else
    {
        last->next = p;
        last = p;
    }
    printf("Элемент '%s' добавлен в очередь\n", p->inf);
}

void dequeue(void)
{
    if (head == NULL)
    {
        printf("Очередь пуста\n");
        return;
    }

    struct node* temp = head;

    if (head == last)
    {
        head = NULL;
        last = NULL;
    }
    else
    {
        head = head->next;
    }

    printf("Элемент '%s' удален из очереди\n", temp->inf);
    free(temp);
}

void review(void)
{
    struct node* struc = head;
    if (head == NULL)
    {
        printf("Очередь пуста\n");
        return;
    }

    printf("Содержимое очереди (первый - будет удален следующим):\n");
    int position = 1;

```

```

while (struc)
{
    printf("%d. Имя - %s\n", position, struc->inf);
    struc = struc->next;
    position++;
}

}

struct node* find(char* name)
{
    struct node* struc = head;
    if (head == NULL)
    {
        printf("Очередь пуста\n");
        return NULL;
    }

    int position = 1;
    while (struc)
    {
        if (strcmp(name, struc->inf) == 0)
        {
            printf("Элемент '%s' найден на позиции %d\n", name, position);
            return struc;
        }
        struc = struc->next;
        position++;
    }

    printf("Элемент '%s' не найден\n", name);
    return NULL;
}

void del(char* name)
{
    struct node* struc = head;
    struct node* prev = NULL;
    int flag = 0;

    if (head == NULL)
    {
        printf("Очередь пуста\n");
        return;
    }

    if (strcmp(name, struc->inf) == 0)
    {
        flag = 1;
        head = struc->next;
        if (head == NULL)
        {
            last = NULL;
        }
        printf("Элемент '%s' удален\n", struc->inf);
        free(struc);
        return;
    }

    prev = struc;
    struc = struc->next;

    while (struc)
    {
        if (strcmp(name, struc->inf) == 0)
        {
            flag = 1;
            prev->next = struc->next;

```

```

        if (struc == last)
        {
            last = prev;
        }

        printf("Элемент '%s' удален\n", struc->inf);
        free(struc);
        return;
    }
    prev = struc;
    struc = struc->next;
}

if (flag == 0)
{
    printf("Элемент '%s' не найден\n", name);
}
}

void peek(void)
{
    if (head == NULL)
    {
        printf("Очередь пуста\n");
        return;
    }

    printf("Первый элемент в очереди: %s\n", head->inf);
}

void is_empty(void)
{
    if (head == NULL)
    {
        printf("Очередь пуста\n");
    }
    else
    {
        printf("Очередь не пуста\n");
    }
}

void clear_queue(void)
{
    struct node* current = head;
    struct node* next;

    while (current != NULL)
    {
        next = current->next;
        free(current);
        current = next;
    }

    head = NULL;
    last = NULL;
    printf("Очередь очищена\n");
}

int main()
{
    system("chcp 1251");
    int choice;
    char name[256];

    printf("Реализация очереди (FIFO)\n");

```

```

while (1)
{
    printf("\nМеню:\n");
    printf("1. Добавить элемент в очередь \n");
    printf("2. Удалить элемент из очереди \n");
    printf("3. Просмотреть очередь\n");
    printf("4. Найти элемент по имени\n");
    printf("5. Удалить элемент по имени\n");
    printf("6. Показать первый элемент\n");
    printf("7. Проверить пустоту очереди\n");
    printf("8. Очистить очередь\n");
    printf("9. Выход\n");
    printf("Выберите действие: ");
    scanf("%d", &choice);

    switch (choice)
    {
        case 1:
            enqueue();
            break;
        case 2:
            dequeue();
            break;
        case 3:
            review();
            break;
        case 4:
            printf("Введите имя для поиска: ");
            scanf("%s", name);
            find(name);
            break;
        case 5:
            printf("Введите имя для удаления: ");
            scanf("%s", name);
            del(name);
            break;
        case 6:
            peek();
            break;
        case 7:
            is_empty();
            break;
        case 8:
            clear_queue();
            break;
        case 9:
            clear_queue();
            printf("Выход из программы\n");
            return 0;
        default:
            printf("Неверный выбор\n");
    }
}

return 0;
}

```

Результат работы программы

Результат работы программы показан на рисунках 2.1 – 2.4

```
Текущая кодовая страница: 1251
Реализация очереди (FIFO)

Меню:
1. Добавить элемент в очередь
2. Удалить элемент из очереди
3. Просмотреть очередь
4. Найти элемент по имени
5. Удалить элемент по имени
6. Показать первый элемент
7. Проверить пустоту очереди
8. Очистить очередь
9. Выход
Выберите действие: 1
Введите название объекта:
Первый
Элемент 'Первый' добавлен в очередь

Меню:
1. Добавить элемент в очередь
2. Удалить элемент из очереди
3. Просмотреть очередь
4. Найти элемент по имени
5. Удалить элемент по имени
6. Показать первый элемент
7. Проверить пустоту очереди
8. Очистить очередь
9. Выход
Выберите действие: 1
Введите название объекта:
Второй
Элемент 'Второй' добавлен в очередь
```

Рисунок 2.1

```
Меню:
1. Добавить элемент в очередь
2. Удалить элемент из очереди
3. Просмотреть очередь
4. Найти элемент по имени
5. Удалить элемент по имени
6. Показать первый элемент
7. Проверить пустоту очереди
8. Очистить очередь
9. Выход
Выберите действие: 3
Содержимое очереди (первый - будет удален следующим):
1. Имя - Первый
2. Имя - Второй
3. Имя - Третий
```

Рисунок 2.2


```
Меню:
1. Добавить элемент в очередь
2. Удалить элемент из очереди
3. Просмотреть очередь
4. Найти элемент по имени
5. Удалить элемент по имени
6. Показать первый элемент
7. Проверить пустоту очереди
8. Очистить очередь
9. Выход
Выберите действие: 6
Первый элемент в очереди: Первый
```

```
Меню:
1. Добавить элемент в очередь
2. Удалить элемент из очереди
3. Просмотреть очередь
4. Найти элемент по имени
5. Удалить элемент по имени
6. Показать первый элемент
7. Проверить пустоту очереди
8. Очистить очередь
9. Выход
Выберите действие: 4
Введите имя для поиска: Первый
Элемент 'Первый' найден на позиции 1
```

Рисунок 2.3

```
Меню:
1. Добавить элемент в очередь
2. Удалить элемент из очереди
3. Просмотреть очередь
4. Найти элемент по имени
5. Удалить элемент по имени
6. Показать первый элемент
7. Проверить пустоту очереди
8. Очистить очередь
9. Выход
Выберите действие: 7
Очередь не пуста
```

```
Меню:
1. Добавить элемент в очередь
2. Удалить элемент из очереди
3. Просмотреть очередь
4. Найти элемент по имени
5. Удалить элемент по имени
6. Показать первый элемент
7. Проверить пустоту очереди
8. Очистить очередь
9. Выход
Выберите действие: 8
Очередь очищена
```

```
Меню:
1. Добавить элемент в очередь
2. Удалить элемент из очереди
3. Просмотреть очередь
4. Найти элемент по имени
5. Удалить элемент по имени
6. Показать первый элемент
7. Проверить пустоту очереди
8. Очистить очередь
9. Выход
Выберите действие: 7
Очередь пуста
```

Рисунок 2.4

Задание 3

```
#define _CRT_SECURE_NO_WARNINGS
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

struct node {
    char inf[256];
    struct node* next;
};

struct node* top = NULL;
char find_el[256];

struct node* get_struct(void);
struct node* find(char* name);
void push(void);
void pop(void);
void review(void);
void del(char* name);

struct node* get_struct(void)
{
    struct node* p = NULL;
    char s[256];

    if ((p = (struct node*)malloc(sizeof(struct node))) == NULL)
    {
        printf("Ошибка при распределении памяти\n");
        exit(1);
    }

    printf("Введите название объекта: \n");
    scanf("%s", s);
    if (*s == 0)
    {
        printf("Запись не была произведена\n");
        free(p);
        return NULL;
    }
    strcpy(p->inf, s);
    p->next = NULL;

    return p;
}

void push(void)
{
    struct node* p = NULL;
    p = get_struct();

    if (p == NULL) {
        return;
    }

    if (top == NULL)
    {
        top = p;
    }
    else
    {
        p->next = top;
        top = p;
    }
    printf("Элемент '%s' добавлен в стек\n", p->inf);
}
```

```

void pop(void)
{
    if (top == NULL)
    {
        printf("Стек пуст\n");
        return;
    }

    struct node* temp = top;
    top = top->next;

    printf("Элемент '%s' удален из стека\n", temp->inf);
    free(temp);
}

void review(void)
{
    struct node* struc = top;
    if (top == NULL)
    {
        printf("Стек пуст\n");
        return;
    }

    printf("Содержимое стека (вершина - будет удалена следующей):\n");
    int position = 1;
    while (struc)
    {
        printf("%d. Имя - %s\n", position, struc->inf);
        struc = struc->next;
        position++;
    }
}

struct node* find(char* name)
{
    struct node* struc = top;
    if (top == NULL)
    {
        printf("Стек пуст\n");
        return NULL;
    }

    int position = 1;
    while (struc)
    {
        if (strcmp(name, struc->inf) == 0)
        {
            printf("Элемент '%s' найден на позиции %d от вершины\n", name,
position);
            return struc;
        }
        struc = struc->next;
        position++;
    }

    printf("Элемент '%s' не найден\n", name);
    return NULL;
}

void del(char* name)
{
    struct node* struc = top;
    struct node* prev = NULL;
    int flag = 0;

```

```

if (top == NULL)
{
    printf("Стек пуст\n");
    return;
}

if (strcmp(name, struc->inf) == 0)
{
    flag = 1;
    top = struc->next;
    printf("Элемент '%s' удален из стека\n", struc->inf);
    free(struc);
    return;
}

prev = struc;
struc = struc->next;

while (struc)
{
    if (strcmp(name, struc->inf) == 0)
    {
        flag = 1;
        prev->next = struc->next;

        printf("Элемент '%s' удален из стека\n", struc->inf);
        free(struc);
        return;
    }
    prev = struc;
    struc = struc->next;
}

if (flag == 0)
{
    printf("Элемент '%s' не найден\n", name);
}
}

void peek(void)
{
    if (top == NULL)
    {
        printf("Стек пуст\n");
        return;
    }

    printf("Вершина стека: %s\n", top->inf);
}

void is_empty(void)
{
    if (top == NULL)
    {
        printf("Стек пуст\n");
    }
    else
    {
        printf("Стек не пуст\n");
    }
}

void clear_stack(void)
{
    struct node* current = top;
    struct node* next;

```

```

while (current != NULL)
{
    next = current->next;
    free(current);
    current = next;
}

top = NULL;
printf("Стек очищен\n");
}

void stack_size(void)
{
    struct node* struc = top;
    int count = 0;

    while (struc)
    {
        count++;
        struc = struc->next;
    }

    printf("Количество элементов в стеке: %d\n", count);
}

int main()
{
    system("chcp 1251");
    int choice;
    char name[256];

    printf("Реализация стека (LIFO)\n");

    while (1)
    {
        printf("\nМеню:\n");
        printf("1. Добавить элемент в стек (push)\n");
        printf("2. Удалить элемент из стека (pop)\n");
        printf("3. Просмотреть стек\n");
        printf("4. Найти элемент по имени\n");
        printf("5. Удалить элемент по имени\n");
        printf("6. Показать вершину стека\n");
        printf("7. Проверить пустоту стека\n");
        printf("8. Размер стека\n");
        printf("9. Очистить стек\n");
        printf("10. Выход\n");
        printf("Выберите действие: ");
        scanf("%d", &choice);

        switch (choice)
        {
            case 1:
                push();
                break;
            case 2:
                pop();
                break;
            case 3:
                review();
                break;
            case 4:
                printf("Введите имя для поиска: ");
                scanf("%s", name);
                find(name);
                break;
            case 5:
                printf("Введите имя для удаления: ");

```

```

        scanf("%s", name);
        del(name);
        break;
    case 6:
        peek();
        break;
    case 7:
        is_empty();
        break;
    case 8:
        stack_size();
        break;
    case 9:
        clear_stack();
        break;
    case 10:
        clear_stack();
        printf("Выход из программы\n");
        return 0;
    default:
        printf("Неверный выбор\n");
    }
}

return 0;
}

```

Результат работы программы

Результат работы программы показан на рисунках 3.1 – 3.3

```
Текущая кодовая страница: 1251
Реализация стека (LIFO)

Меню:
1. Добавить элемент в стек
2. Удалить элемент из стека
3. Просмотреть стек
4. Найти элемент по имени
5. Удалить элемент по имени
6. Показать вершину стека
7. Проверить пустоту стека
8. Размер стека
9. Очистить стек
10. Выход
Выберите действие: 1
Введите название объекта:
Первый
Элемент 'Первый' добавлен в стек

Меню:
1. Добавить элемент в стек
2. Удалить элемент из стека
3. Просмотреть стек
4. Найти элемент по имени
5. Удалить элемент по имени
6. Показать вершину стека
7. Проверить пустоту стека
8. Размер стека
9. Очистить стек
10. Выход
Выберите действие: 1
Введите название объекта:
Второй
Элемент 'Второй' добавлен в стек

Меню:
1. Добавить элемент в стек
2. Удалить элемент из стека
3. Просмотреть стек
4. Найти элемент по имени
5. Удалить элемент по имени
6. Показать вершину стека
7. Проверить пустоту стека
8. Размер стека
9. Очистить стек
10. Выход
Выберите действие: 1
Введите название объекта:
Третий
```

Рисунок 3.1

```
Меню:
1. Добавить элемент в стек
2. Удалить элемент из стека
3. Просмотреть стек
4. Найти элемент по имени
5. Удалить элемент по имени
6. Показать вершину стека
7. Проверить пустоту стека
8. Размер стека
9. Очистить стек
10. Выход
Выберите действие: 3
Содержимое стека (вершина - будет удалена следующей):
1. Имя - Третий
2. Имя - Второй
3. Имя - Первый

Меню:
1. Добавить элемент в стек
2. Удалить элемент из стека
3. Просмотреть стек
4. Найти элемент по имени
5. Удалить элемент по имени
6. Показать вершину стека
7. Проверить пустоту стека
8. Размер стека
9. Очистить стек
10. Выход
Выберите действие: 2
Элемент 'Третий' удален из стека
```

Рисунок 3.2

```
Меню:
1. Добавить элемент в стек
2. Удалить элемент из стека
3. Просмотреть стек
4. Найти элемент по имени
5. Удалить элемент по имени
6. Показать вершину стека
7. Проверить пустоту стека
8. Размер стека
9. Очистить стек
10. Выход
Выберите действие: 6
Вершина стека: Второй

Меню:
1. Добавить элемент в стек
2. Удалить элемент из стека
3. Просмотреть стек
4. Найти элемент по имени
5. Удалить элемент по имени
6. Показать вершину стека
7. Проверить пустоту стека
8. Размер стека
9. Очистить стек
10. Выход
Выберите действие: 8
Количество элементов в стеке: 2
```

Рисунок 3.3

Вывод: В ходе выполнения лабораторной работы были успешно освоены принципы работы с динамическими структурами данных на примере односвязных списков.